

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumnos – Grupo Nro.368

Colazo Sebastián – Comisión Nro.3

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación 1

Docente Titular

Ariel Enferrel

Docente Tutor

Matías Torres

25 de junio de 2026

Tabla de contenido

No se encontraron entradas de tabla de contenido.

1.Marco teórico e investigación

1.1 Estructuras de Datos Nativas: Listas y Diccionarios

En el ámbito de la ciencia de la computación, una estructura de datos es una forma particular de organizar y almacenar información en la memoria de un ordenador para que pueda ser utilizada de manera eficiente. En este proyecto se combinaron dos estructuras dinámicas esenciales de Python: las listas y los diccionarios.

- **Diccionarios (dict):** Un diccionario es una colección mutable, desordenada (en versiones clásicas) e indexada por claves (*keys*). A diferencia de las secuencias tradicionales que utilizan índices numéricos, los diccionarios asocian una clave única a un valor determinado (mapeo *key-value*).
 - *Aplicación en el proyecto:* Cada país del dataset se modeló de forma independiente como un diccionario. Las propiedades de la entidad geográfica se definieron mediante claves de tipo cadena: 'nombre', 'poblacion', 'superficie' y 'continente'. Esta abstracción semántica permite acceder a los atributos numéricos de un país de manera directa (por ejemplo, pais['poblacion']), lo que dota al programa de mayor tolerancia a fallos en comparación con el uso de tuplas o listas indexadas por posición, donde alterar el orden de los campos rompería la lógica del software.
- **Listas (list):** Una lista es una estructura de datos lineal, mutable y ordenada que permite almacenar secuencias de elementos de cualquier tipo de datos.
 - *Aplicación en el proyecto:* Se utilizó una lista global para contener todos los diccionarios de los países cargados desde el almacenamiento secundario. Debido a que las listas en Python manejan la asignación de memoria de manera dinámica, facilitan la expansión del dataset en tiempo de ejecución (mediante el método `.append()`) cuando el usuario registra un nuevo país, permitiendo además realizar búsquedas repetitivas e iteraciones vectoriales eficientes.

1.2 Modularización y Estructura de Funciones

La modularización es el proceso de dividir un sistema de software en partes diferenciadas llamadas módulos o funciones, las cuales deben ser lo más independientes posible entre sí (alta cohesión y bajo acoplamiento).

- **Principio de Única Responsabilidad:** En el diseño del caso práctico, se aplicó la modularización dividiendo el entorno en dos subcapas físicas: `funciones.py` (lógica de negocio y computación) y `main.py` (interfaz de usuario). Cada función dentro de `funciones.py` fue diseñada bajo el principio de resolver una única tarea (por ejemplo, aislar el algoritmo analítico en `calcular_estadisticas()`). Esto previene la duplicación de código, simplifica la depuración de errores lógicos y permite que el menú principal invoque procesos estructurados sin conocer la complejidad interna del algoritmo de cómputo.

1.3 Estructuras de Control de Flujo

El flujo de ejecución de un programa estructurado está determinado por instrucciones que evalúan predicados lógicos o repiten secuencias de comandos.

- **Estructuras Repetitivas (`while` y `for`):** El bucle iterativo condicional `while True` en `main.py` sostiene la persistencia de la interfaz en consola, asegurando que el sistema permanezca en ejecución activa hasta que ocurra una interrupción explícita (Opción 8). Por otro lado, los bucles definidos `for` se emplean para recorrer secuencialmente la lista de países, permitiendo evaluar individualmente cada diccionario en búsquedas de subcadenas o acumulaciones matemáticas.
- **Estructuras Condicionales (`if-elif-else`):** Actúan como bifurcaciones lógicas basadas en el álgebra de Boole. En el sistema, controlan el enrutamiento de las opciones del menú y realizan validaciones críticas de datos (por ejemplo, comprobar si una cadena está vacía o si un valor numérico es menor o igual a cero antes de insertarlo en la base de datos).

1.4 Algoritmos de Filtrado y Ordenamiento

- **Filtrado:** Es la operación de discriminar elementos de una colección basándose en si cumplen o no una condición lógica (predicado). El motor de filtrado del sistema utiliza estructuras condicionales anidadas dentro de bucles para evaluar

rangos cuantitativos (mínimo $\leq x \leq$ máximo) o coincidencias cualitativas (continente exacto), construyendo sublistas estables con los registros aprobados.

- **Ordenamiento Dinámico con Expresiones Lambda:** El ordenamiento es el proceso de organizar los elementos de una lista en una secuencia determinada (ascendente o descendente) según un criterio específico. Para evitar la programación manual de algoritmos de ordenamiento ineficientes (como el método de la burbuja), se utilizó la función nativa `sorted()` combinada con expresiones **lambda** (funciones anónimas de una sola línea). Al pasar una función lambda como parámetro de clave (`key=lambda x: x[clave_ordenamiento]`), se le instruye al intérprete de Python qué propiedad del diccionario de países debe extraer para indexar y ordenar el vector, optimizando drásticamente el rendimiento en memoria RAM.

1.5 Persistencia de Datos en Archivos CSV

Los datos almacenados en variables dentro de la memoria RAM son volátiles y se destruyen al cerrarse el proceso del programa. Para garantizar la **persistencia** (la supervivencia de la información en el tiempo), el sistema implementa mecanismos de entrada y salida (*I/O*) de archivos.

- El formato **CSV** (*Comma-Separated Values*) representa datos tabulares en archivos de texto plano separados por un delimitador (comas). Mediante el uso de la librería estándar `csv` de Python, las funciones `DictReader` y `DictWriter` actúan como un puente de persistencia: parsean e interpretan las líneas del archivo de texto transformándolas en diccionarios de Python en la inicialización, y serializan los diccionarios de regreso a texto plano estructurado al guardar los cambios, protegiendo la integridad de la base de datos local.

2.Objetivo del trabajo

El objetivo principal de este proyecto integrador es el diseño, desarrollo e implementación de una aplicación de consola en Python destinada a la gestión, administración y análisis de un dataset geográfico enfocado en indicadores globales de países.

A través de este sistema, se busca construir una herramienta interactiva y robusta que permita centralizar operaciones esenciales como el registro de nuevas entidades, la

modificación controlada de sus atributos numéricos, búsquedas avanzadas por coincidencia y la generación de métricas estadísticas complejas en tiempo de ejecución.

El desarrollo persigue dos metas arquitectónicas clave:

1. **Persistencia Eficiente:** Integrar de manera limpia los datos con almacenamiento secundario mediante archivos estructurados en formato CSV, garantizando que toda la manipulación de la información en la memoria RAM (a través de estructuras dinámicas de listas y diccionarios) se preserve de manera consistente al cerrarse la sesión.
2. **Robustez y Tolerancia a Fallos:** Diseñar una interfaz interactiva de consola que implemente un control estricto de excepciones (try-except) y validaciones preventivas, evitando interrupciones inesperadas del programa (*crashes*) ante el ingreso de tipos de datos inválidos o registros duplicados por parte del usuario.

3. Diseño del caso práctico

El diseño del sistema se basó en los principios de la programación estructurada y la modularización, buscando una separación clara entre la interfaz de usuario y el motor de procesamiento lógico. A continuación, se detalla la arquitectura de archivos del software y el modelado conceptual de la información.

3.1 Arquitectura de Módulos del Sistema

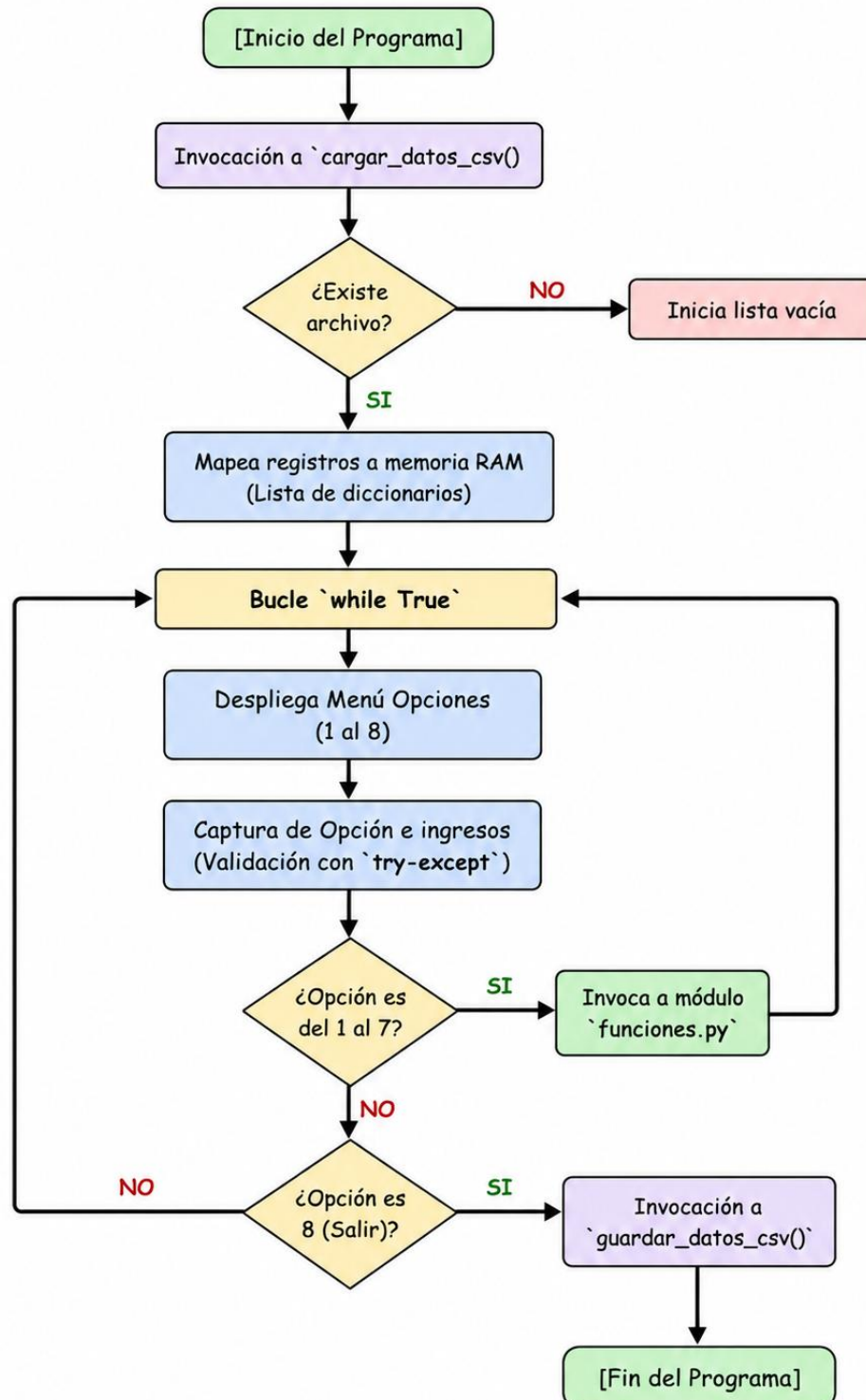
El código fuente se fragmentó en archivos independientes con responsabilidades unívocas y bien delimitadas, logrando un esquema de alta cohesión y bajo acoplamiento:

- **países.csv (Capa de Almacenamiento):** Archivo físico de texto plano que actúa como base de datos persistente no volátil, estructurada mediante cabeceras separadas por comas (nombre,poblacion,superficie,continente).
- **funciones.py (Capa de Lógica de Negocio):** Módulo encargado exclusivamente de la manipulación algorítmica de las estructuras de datos. No contiene instrucciones de entrada (input()) ni de salida directa en pantalla (print()) destinadas al usuario general, aislando los procesos de ordenamiento, filtrado, lectura/escritura y cómputo de métricas estadísticas.
- **main.py (Capa de Presentación / Interfaz):** Orquestador central de la aplicación. Contiene el bucle interactivo de consola, despliega el menú de

opciones, captura las entradas del teclado y gestiona el control preventivo de errores y excepciones (try-except).

3.2 Modelado y Diagrama de Flujo de Operaciones

El ciclo de ejecución sigue una secuencia lineal de inicialización y un ciclo iterativo para las consultas.



3.3 Representación Conceptual en Memoria RAM

Para garantizar que el motor de software sea flexible y escalable, el dataset en memoria RAM fue diseñado imitando la estructura de una base de datos relacional tabular:

- La **tabla completa** se almacena en una variable de tipo **Lista** (un vector indexado numéricamente).
- Cada **fila o registro** se abstrae como una entidad independiente de tipo **Diccionario**, mapeando propiedades fijas a través de claves literales.

Estructura en Memoria:

```
lista_paises = [  
    { "nombre": "Argentina", "poblacion": 45376763, "superficie": 2780400,  
      "continente": "América" },  
    { "nombre": "Japón", "poblacion": 125800000, "superficie": 377975, "continente":  
      "Asia" },  
    ...  
]
```

4. Metodología utilizada

Para asegurar la correcta ejecución del proyecto y el cumplimiento de todos los requerimientos en los plazos establecidos, se adoptó una metodología de desarrollo de software de carácter **incremental, modular y orientada a pruebas continuas**. Esta estrategia técnica se desglosa en las siguientes etapas secuenciales:

4.1 Planificación y Descomposición del Problema

El proceso comenzó con un análisis del dominio (dataset de países) y las restricciones impuestas por la cátedra (campos obligatorios, prohibición de registros vacíos y validación de rangos numéricos positivos). El problema principal se descompuso utilizando un enfoque *Top-Down*, aislando cada requerimiento funcional mínimo como una subtarea independiente antes de escribir la primera línea de código.

4.2 Codificación Incremental por Capas de Aislamiento

El desarrollo del código no se realizó de forma masiva, sino incremental:

- **Fase de Persistencia:** Primero se programaron y consolidaron de forma exclusiva las funciones de lectura y escritura del archivo plano (`cargar_datos_csv` y `guardar_datos_csv`), garantizando la correcta traducción de filas de texto a estructuras de diccionarios en memoria RAM.
- **Fase de Operaciones CRUD y Motores Analíticos:** Una vez estabilizada la base de datos local, se programaron individualmente los módulos lógicos en `funciones.py` (altas, modificaciones, algoritmos de ordenamiento `lambda`, búsquedas y cálculos estadísticos).
- **Fase de Construcción de Interfaz:** Finalmente, se realizó el menú en `main.py`, conectando la entrada del usuario por teclado con las funciones lógicas previamente probadas.

4.3 Plan de Testeo Iterativo y Control de Excepciones

Se aplicó un testeo dinámico durante cada fase de la codificación.

- **Pruebas de Caja Blanca:** Se realizaron ejecuciones parciales de cada función matemática y lógica para verificar la exactitud de los promedios globales y los extremos (máximos y mínimos) de población.
- **Inyección de Errores Preventiva:** Se forzó intencionalmente la inserción de caracteres alfabéticos en campos que requerían números enteros (Población y Superficie), lo que permitió identificar vulnerabilidades de tipo `ValueError`. A raíz de esto, se implementaron estructuras de captura de excepciones (`try-except`) en el menú interactivo para interceptar los fallos y desplegar mensajes limpios de advertencia, asegurando la continuidad del programa.

4.4 Gestión del Repositorio y Entorno de Desarrollo

Para asegurar la consistencia y la entrega transparente del software, se utilizó el sistema de control de versiones **Git** y la plataforma **GitHub**. Los archivos se estructuraron de manera limpia en la raíz del repositorio público para facilitar una revisión directa del código fuente por parte del cuerpo docente.

5.Resultados Obtenidos

En esta sección se exponen las evidencias del funcionamiento del sistema de gestión geográfica, recopiladas a partir de las pruebas de ejecución en la terminal de comandos. Los resultados demuestran el cumplimiento estricto de los requerimientos de diseño, modularización y tolerancia a fallos exigidos por la cátedra.

5.1 Inicialización y Carga Automatizada del Dataset

Al lanzar la aplicación mediante el comando `python main.py`, el sistema invoca inmediatamente la rutina de persistencia en memoria RAM. En caso de detectar la ausencia física del archivo local, el software activa su lógica defensiva inicializando una estructura limpia y generando un nuevo contenedor estructurado con las cabeceras correspondientes. Al encontrarse el archivo con registros, despliega una interfaz numerada limpia.

```
-----  
SISTEMA DE GESTIÓN DE PAÍSES (TPI)  
1. Mostrar todos los países  
2. Agregar un nuevo país  
3. Actualizar Población y Superficie de un país  
4. Buscar país por nombre (parcial o exacto)  
5. Filtrar países (Continente / Población / Superficie)  
6. Ordenar países (Nombre / Población / Superficie)  
7. Mostrar estadísticas generales  
8. Guardar cambios y Salir  
Seleccione una opción (1-8): █
```

5.2 Presentación Tabular de Datos (Opción 1)

Al seleccionar la Opción 1, la aplicación procesa la lista de diccionarios en memoria RAM. Mediante el uso de f-strings y formateadores de ancho fijo, el sistema alinea la información en columnas legibles. Las variables cuantitativas (Población y Superficie) incluyen especificaciones de formato que aplican automáticamente separadores de miles, asegurando una estética profesional en la consola.

Seleccione una opción (1-8): 1
LISTADO COMPLETO DE PAÍSES

Nombre	Población	Superficie (km ²)	Continente
Argentina	45,376,763	2,780,400	América
Japón	125,800,000	377,975	Asia
Brasil	213,993,437	8,515,767	América
Alemania	83,149,300	357,022	Europa
Francia	67,390,000	551,695	Europa
Sudáfrica	59,310,000	1,221,037	África
Australia	25,690,000	7,692,024	Oceanía

5.3 Registro y Robustez ante Errores de Inserción (Opción 2)

El módulo de altas permite incorporar nuevos países garantizando que no existan nombres duplicados (mediante comparaciones normalizadas en minúsculas). La robustez del sistema se evidencia al forzar ingresos erróneos: ante la introducción de caracteres alfabéticos en los campos numéricos de población o superficie, el bloque try-except captura la excepción ValueError, despliega un aviso preventivo y preserva la ejecución del bucle sin colapsar

```

Seleccione una opción (1-8): 2
REGISTRAR NUEVO PAÍS
Nombre del país: Francia
Continente: Europa
Población (habitantes): 5
Error: La población y la superficie deben ser números enteros válidos.
SISTEMA DE GESTIÓN DE PAÍSES (TPI)
1. Mostrar todos los países
2. Agregar un nuevo país
3. Actualizar Población y Superficie de un país
4. Buscar país por nombre (parcial o exacto)
5. Filtrar países (Continente / Población / Superficie)
6. Ordenar países (Nombre / Población / Superficie)
7. Mostrar estadísticas generales
8. Guardar cambios y Salir
Seleccione una opción (1-8): █

```

```

Seleccione una opción (1-8): 2
REGISTRAR NUEVO PAÍS
Nombre del país: Chile
Continente: América
Población (habitantes): 1900
Superficie (km2): 2500
Éxito: 'Chile' ha sido agregado correctamente.

```

Seleccione una opción (1-8): 1
LISTADO COMPLETO DE PAÍSES

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América
Japón	125,800,000	377,975	Asia
Brasil	213,993,437	8,515,767	América
Alemania	83,149,300	357,022	Europa
Francia	67,390,000	551,695	Europa
Sudáfrica	59,310,000	1,221,037	África
Australia	25,690,000	7,692,024	Oceanía
Chile	1,900	2,500	América

5.4 Modificación Dinámica en Memoria (Opción 3)

La opción de actualización solicita al usuario el nombre del registro a modificar. Al localizarlo de manera secuencial, el sistema permite sobrecribir los atributos cuantitativos directamente sobre las claves del diccionario específico en la memoria RAM, quedando disponibles inmediatamente para el resto de las consultas de la sesión.

```

Seleccione una opción (1-8): 3
ACTUALIZAR DATOS DE UN PAÍS
Ingrese el nombre exacto del país a modificar: Chile
Nueva población: 25000
Nueva superficie en km²: 10
Éxito: Se actualizaron los datos de 'Chile'.
  
```

Seleccione una opción (1-8): 1
LISTADO COMPLETO DE PAÍSES

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América
Japón	125,800,000	377,975	Asia
Brasil	213,993,437	8,515,767	América
Alemania	83,149,300	357,022	Europa
Francia	67,390,000	551,695	Europa
Sudáfrica	59,310,000	1,221,037	África
Australia	25,690,000	7,692,024	Oceanía
Chile	25,000	10	América

5.5 Búsqueda por Coincidencia Parcial de Subcadenas (Opción 4)

El motor de búsqueda demuestra su flexibilidad al utilizar el operador `in` combinado con el método `.lower()`. Esto habilita búsquedas de coincidencia parcial que aíslan los

registros que contienen la subcadena ingresada, eliminando la rigidez de mayúsculas o minúsculas.

Seleccione una opción (1-8): 4
 BUSCAR PAÍS POR NOMBRE
 Ingrese el nombre o texto a buscar: Ar

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América

5.6 Filtrado Intervalar de Datos (Opción 5)

El sistema ejecuta con éxito la segmentación del dataset mediante la evaluación de rangos numéricos inclusivos (minimo $\leq$ valor $\leq$ maximo). El flujo de control valida previamente que el límite máximo no sea inferior al mínimo, garantizando la consistencia lógica antes de procesar el bucle de filtrado en funciones.py.

Seleccione una opción (1-8): 5
 FILTRAR PAÍSES
 1. Por Continente
 2. Por Rango de Población
 3. Por Rango de Superficie
 Seleccione el criterio de filtro (1-3): 3
 Ingrese el valor MÍNIMO: 100000
 Ingrese el valor MÁXIMO: 5000000

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América
Japón	125,800,000	377,975	Asia
Alemania	83,149,300	357,022	Europa
Francia	67,390,000	551,695	Europa
Sudáfrica	59,310,000	1,221,037	África

Seleccione una opción (1-8): 5
 FILTRAR PAÍSES
 1. Por Continente
 2. Por Rango de Población
 3. Por Rango de Superficie
 Seleccione el criterio de filtro (1-3): 1
 Ingrese el nombre del continente: América

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América
Brasil	213,993,437	8,515,767	América
Chile	25,000	10	América

5.7 Ordenamiento Eficiente mediante Expresiones Lambda

(Opción 6)

Al solicitar el ordenamiento del dataset, el programa mapea las opciones del usuario y ejecuta la función nativa `sorted()` utilizando una función anónima lambda como clave dinámica. Los resultados en la terminal reflejan la reestructuración exacta de la grilla en sentido ascendente o descendente según el criterio elegido.

```
Seleccione una opción (1-8): 6
```

```
ORDENAR PAÍSES
```

```
1. Ordenar por Nombre
```

```
2. Ordenar por Población
```

```
3. Por Superficie
```

```
Seleccione la clave de ordenamiento (1-3): 1
```

```
¿Desea el orden descendente? (S/N): S
```

Nombre	Población	Superficie (km²)	Continente

Sudáfrica	59,310,000	1,221,037	África
Japón	125,800,000	377,975	Asia
Francia	67,390,000	551,695	Europa
Chile	25,000	10	América
Brasil	213,993,437	8,515,767	América
Australia	25,690,000	7,692,024	Oceanía
Argentina	45,376,763	2,780,400	América
Alemania	83,149,300	357,022	Europa

5.8 Procesamiento Estadístico y Persistencia Final (Opciones 7 y

8)

La opción analítica realiza un recorrido único optimizado sobre la estructura mutable para calcular promedios flotantes, identificar extremos absolutos (máximos y mínimos) y computar frecuencias por continente. Finalmente, al presionar la Opción 8, el bucle interactivo se rompe de forma segura e invoca a `csv.DictWriter`, ejecutando el volcado físico que consolida de manera permanente todos los cambios en el disco.

Seleccione una opción (1-8): 7
ESTADÍSTICAS DEL DATASET
País con MAYOR población: Brasil (213,993,437 hab.)
País con MENOR población: Chile (25,000 hab.)
Promedio de población global: 77,591,812.50 habitantes
Promedio de superficie global: 2,686,991.25 km²
Cantidad de países por Continente:
• América: 3
• Asia: 1
• Europa: 2
• África: 1
• Oceanía: 1

6.Dificultades Técnicas y Soluciones

6.1 Vulnerabilidad ante Datos de Entrada Incoherentes (Control de Errores)

- **Dificultad:** El programa sufría fallos críticos y salidas abruptas de consola (crashes) por excepciones de tipo ValueError cuando el usuario ingresaba de manera accidental caracteres alfabéticos o cadenas vacías en campos que requerían estrictamente datos numéricos (como población, superficie o rangos de filtrado).
- **Solución:** Se implementó una arquitectura de programación defensiva mediante el uso extensivo de bloques de control de excepciones try-except. De esta manera, cualquier entrada inválida es interceptada a nivel de interfaz en la función de control de flujo. El sistema mitiga el error lanzando una advertencia limpia en pantalla y, apoyado en el bucle continuo while True, redirige al usuario de forma segura al menú principal sin interrumpir el proceso de la aplicación.

6.2 Flexibilidad en Algoritmos de Búsqueda y Ordenamiento

- **Dificultad:** Exigir coincidencias exactas y sensibles a mayúsculas/minúsculas volvía el motor de búsquedas rígido y propenso a simular la ausencia de registros de manera errónea ante sutiles variaciones de tipeo. Por otro lado, diseñar

algoritmos de ordenamiento manuales (como el ordenamiento de burbuja) penalizaba el rendimiento del sistema.

- **Solución:** Para las búsquedas se normalizaron las entradas aplicando sistemáticamente el método `.lower()` en ambos extremos de la evaluación, combinándolo con el operador de membresía `in` para alcanzar búsquedas por coincidencia parcial de subcadenas. Para el ordenamiento, se reemplazó cualquier esquema manual ineficiente por la función nativa de orden superior `sorted()`, parametrizada mediante una expresión lambda que actúa como clave dinámica para resolver de forma limpia y eficiente la indexación por cualquier atributo del diccionario.

7.Conclusión

Con este trabajo aprendí a juntar la programación con el uso de archivos de texto guardados en la computadora. También vi lo importante que es proteger el programa para que no se cierre con un cartel de error si me equivoco al escribir un número.

Elegí estas herramientas porque funcionan muy bien juntas: el archivo CSV me sirve para guardar los datos de forma liviana y rápida sin complicarme con programas externos; las listas y diccionarios me permiten guardar y encontrar los datos del país en la memoria de la computadora de forma ordenada; y la función lambda me ayuda a ordenar las tablas al instante y de manera automática.

Por ejemplo, gracias a estas herramientas logré que se pueda buscar un país escribiendo solo una parte del nombre (como poner "ar" para encontrar "Argentina"), que el programa avise con un cartel si meto letras donde van números, y que las cantidades de población se vean prolijas con los puntos de los miles en la pantalla.

Para organizar el trabajo, dividí el código en dos partes lógicas. En un archivo (`funciones.py`) puse todo lo que hace los cálculos y maneja los datos. En el otro archivo (`main.py`) armé el menú de opciones y los carteles que ve el usuario. Esto me permitió probar cada función por separado y lograr que el código final quede ordenado y sea fácil de entender.

8. Links de interés

Link del repositorio de GitHub: <https://github.com/sebascolazo20/TPI-Programacion.git>

Link del video explicativo: <https://www.youtube.com/watch?v=126bU2xByQA>

9. Bibliografía

- **Python Software Foundation (2026).** *The Python Standard Library - csv File Reading and Writing* (Manejo y lectura de archivos planos estructurados CSV):
<https://docs.python.org/3/library/csv.html>
- **Python Software Foundation (2026).** *Built-in Functions: sorted()* (Algoritmia de ordenamiento nativo y optimización de búsquedas):
<https://docs.python.org/3/library/functions.html#sorted>
- **Python Software Foundation (2026).** *Data Structures - Lists and Dictionaries* (Estructuras de datos dinámicas, mutabilidad y colecciones de datos en memoria RAM):<https://docs.python.org/3/tutorial/datastructures.html>
- **Python Software Foundation (2026).** *Errors and Exceptions - Handling Exceptions* (Gestión defensiva de errores mediante estructuras Try-Except y manejo de ValueError): <https://docs.python.org/3/tutorial/errors.html>
- **Python Software Foundation (2026).** *Functional Programming Modules: Lambda Expressions* (Funciones anónimas aplicadas como claves dinámicas de ordenamiento).
<https://docs.python.org/3/reference/expressions.html#lambda>
- **Van Rossum, G., Warsaw, B., & Coghlan, N. (2013).** *PEP 8 – Style Guide for Python Code* (Buenas prácticas de diseño de código, legibilidad y modularización).:
<https://peps.python.org/pep-0008/>